Specification

Season: 4 · Enterprise · Category: Enterprise · Level: Intermediate

Intent

- Composable business rules as predicates.

Problem

- Without this pattern, code tends toward: New things up with conditionals and duplicated construction.

Real-world Analogy

- Filter listings: price AND in-stock.

When to Use

- Microservices / modular monoliths.
- The design force named in Intent shows up more than once.

When Not to Use

- Greenfield CRUD without distribution.
- A simpler language feature (record, sealed switch, function value) already solves it.

Pattern Structure

- See mermaid diagram in explanation.md

Classic Implementation

```java
package com.buildwithwaqar.patterns.enterprise.specification;

import java.util.function.Predicate;

public interface Specification<T> extends Predicate<T> {
    default Specification<T> and(Specification<T> other) {
        return t -> this.test(t) && other.test(t);
    }

    default Specification<T> or(Specification<T> other) {
        return t -> this.test(t) || other.test(t);
    }

    default Specification<T> not() { return t -> !this.test(t); }
}

record Product(String name, int priceCents, boolean inStock) {}

final class PriceSpec implements Specification<Product> {
    private final int max;
    PriceSpec(int max) { this.max = max; }
    public boolean test(Product p) { return p.priceCents() <= max; }
}

final class InStockSpec implements Specification<Product> {
    public boolean test(Product p) { return p.inStock(); }
}
```

Modern Java 25 / Functional

- Sealed events + pattern matching for projections.

Worked Example

```
See the JUnit test beside the implementation.
Run: mvn -Dtest=*Test test
Focus package matching: enterprise/specification/
```

Pitfalls

- Keep demos minimal — production needs idempotency and observability.
- Saga compensations must be careful.

Interview Q&A

- Q: Production concern for Specification? A: Idempotency, observability, and failure modes.
- Q: How would you test this? A: Prefer injecting interfaces or interpreters; keep demos deterministic (fixed clocks, canned inputs).

Key Takeaways

- Composable business rules as predicates.
- Prefer Java 25 sealed types / records / virtual threads where they simplify the classic shape.
- Ship working tests with every episode.

Video Script Highlights

- [0:00] Hook — Open with the analogy: Filter listings: price AND in-stock. [0:40] Intent — State the design force in one sentence: Composa...